Openi nCol ab

In [ ]:

```
import pandas as pd
import numpy as np
import sklearn as sk
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, accuracy_score, classificati
from sklearn.tree import plot_tree
```

In [ ]:

```
data = pd.read_csv('/content/wireless_signal_quality_dataset.csv')
print("first five records")
print(data.head)
print("\nDataset Shape")
```

```
first five records
<bound method NDFrame.head of   Signal_Strength_dBm SNR_dB Interference_
dB Distance_m Frequency_GHz \
0  1                -41          6                0      257           5.0
2  3                -86         25               20      134           2.4
4  ..               -73         27                0      385           2.4
95                  -57          5                0      255           5.0
96                  -87         15               24       61           5.0
97                  ...        ...              ...      ...           ...
98                  -75         37               11       17           2.4
99                  -88         11               18      231           5.0
                    -57          6               27      460           5.0
0  1                -49         25               24      298           2.4
2  3                -46          8               29      423           5.0
4  ..
95   Channel_Bandwidth_MHz Noise_Floor_dBm Connected_Devices \
96                     160            -100                24
97                     160             -96                41
98                      20             -72                13
99                     160             -86                 9
                        20             -79                32
                       ...             ...               ...
                       160             -80                34
                        20             -90                25
                       160             -76                24
                        20             -78                39
                        40             -71                16

      Estimated_Throughput_Mbps Quality
0  1                         18    Poor
2  3                         55    Poor
4  ..                        81    Fair
95                           15    Poor
96                           21    Poor
97                          ...     ...
98                          100    Fair
99                           15    Poor
                              1    Poor
                             51    Fair
                              1    Poor

[100 rows x 10 columns]>
```

Dataset Shape

In [ ]:
```
x=data.drop('Quality',axis=1)
y=data['Quality']

encoder = LabelEncoder()
y = encoder.fit_transform(y)

print("\nClass Labels")
print(encoder.classes_)
```

Class Labels
['Excellent' 'Fair' 'Good' 'Poor']

In [ ]:
```
print(data.isnull().sum())
```

Signal_Strength_dBm	0
SNR_dB	0
Interference_dB	0
Distance_m	0
Frequency_GHz	0
Channel_Bandwidth_MHz	0
Noise_Floor_dBm	0
Connected_Devices	0
Estimated_Throughput_Mbps	0
Quality	0

dtype: int64

In [ ]:
```
#check file loaded as a 1 column
if len(data.columns) == 1:
    print("incorrect csv format datacted. reparing")
```

In [ ]:
```
data.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	Signal_Strength_dBm	100 non-null	int64
1	SNR_dB	100 non-null	int64
2	Interference_dB	100 non-null	int64
3	Distance_m	100 non-null	int64
4	Frequency_GHz	100 non-null	float64
5	Channel_Bandwidth_MHz	100 non-null	int64
6	Noise_Floor_dBm	100 non-null	int64
7	Connected_Devices	100 non-null	int64
8	Estimated_Throughput_Mbps	100 non-null	int64
9	Quality	100 non-null	object

dtypes: float64(1), int64(8), object(1)
memory usage: 7.9+ KB

In [ ]:
```
data.describe()
```

Out[ ]:

	Signal_Strength_dBm	SNR_dB	Interference_dB	Distance_m	Frequency_GHz	Chann
count	100.000000	100.000000	100.000000	100.000000	100.00000	
mean	-69.020000	21.150000	14.490000	264.500000	4.34400	

min	-95.000000	5.000000	0.000000	17.000000	2.40000
25%	-84.250000	11.750000	7.000000	158.750000	2.40000
50%	-69.500000	19.500000	14.000000	270.500000	5.00000
75%	-55.000000	31.000000	21.250000	370.500000	6.00000
max	-40.000000	40.000000	30.000000	499.000000	6.00000

In [ ]:
```
from re import X
X_train, X_test, y_train, y_test = train_test_split(
      x,
      y,
      test_size=0.25,    random_state=42,
      stratify=y
      )
```

In [ ]:
```
#Train Decision tree
dt = DecisionTreeClassifier(
      criterion="gini",
      max_depth=5,
      random_state=42
)
dt.fit(X_train, y_train)
```

Out[ ]:
DecisionTreeClassifier(max_depth=5, random_state=42)
InaJupyterenvironment,pleasererunthiscelltoshowtheHTMLrepresentation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [ ]:
```
#prediction
y_pread = dt.predict(X_test)
```

In [ ]:
```
#Accuaracy
accuracy = accuracy_score(y_test, y_pread)
print("\nAccuracy: {:.2f}%".format(accuracy*100))
```

Accuracy: 92.00%

In [ ]:
```
#confusion matrix
cm = confusion_matrix(y_test, y_pread)
print("\nConfusion Matrix")
print(cm)
```

Confusion Matrix
[[0 0 1 0]
[0 8 1 0]
[0 0 2 0]
[0 0 0 13]]

In [ ]:
```
#classification report
print("\nClassification Report;")
print(classification_report(y_test, y_pread,target_names=encoder.classes_)
```

Cass cato epot;

	precision	recall	f1-score	support
Excellent	0.00	0.00	0.00	1
Fair	1.00	0.89	0.94	9
Good	0.50	1.00	0.67	2
Poor	1.00	1.00	1.00	13
accuracy			0.92	25
macro avg	0.62	0.72	0.65	25
weighted avg	0.92	0.92	0.91	25

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1 565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0i n labels with no predicted samples. Use `zero_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1 565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0i n labels with no predicted samples. Use `zero_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1 565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0i n labels with no predicted samples. Use `zero_division` parameter to control this behavior.

_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

In [ ]:

```
#Feature importance
importances = pd.Series(
    dt.feature_importances_,
    index=x.columns
)
importace = importances.sort_values(ascending=False)
print(importances)
```

Signal_Strength_dBm 0.438873
SNR_dB 0.365448
Interference_dB 0.078643
Distance_m 0.000000
Frequency_GHz 0.000000